SHETH L.U.J. AND SIR M.V. DEGREE COLLEGE OF SCIENCE

SUBJECT NAME: <u>T101 ARTIFICIAL INTELLIGENCE Practical</u>

<u>PRACTICAL - 1</u>

AIM :-

To implement Breadth First Search (BFS) algorithm for finding a route from Home to College using the available road network. .

Code :-

1. Practical_1 Code.py

```
# T101 RAJDEEP M PARAB
# Breadth First Search (BFS)
# Problem: Find a route from Home to MVLU College
import queue as Q
# ROAD NETWORK
dict_gn = dict(
    Home=dict(
        Matharpada_Rd=110
    ),
    Matharpada_Rd=dict(
        Amit_Naik_Marg=87
    ),
    Amit_Naik_Marg=dict(
        Caesar_Cross_Rd_No_1=110
    ),
    Caesar_Cross_Rd_No_1=dict(
        Caesar_Rd=58,
        Caesar_Rd_Alternative=180
    ),
    # Route 1
    Caesar_Rd=dict(
        Swami_Vivekanand_Rd=240
    ),
    # Route 2
    Caesar_Rd_Alternative=dict(
        Swami_Vivekanand_Rd=650
    ),
    Swami_Vivekanand_Rd=dict(
        Model_Co_op_Bank=400
    ),
    Model_Co_op_Bank=dict(
        Andheri_Subway_Rd=400
    ),
    Andheri_Subway_Rd=dict(
```

```
        Dr_S_Radha_Krishnan_Rd=500
    ),
    Dr_S_Radha_Krishnan_Rd=dict(
        MVLU_College=230
    ),
    MVLU_College=dict()
)

# TWO AVAILABLE ROUTES
route1 = [
    "Home",
    "Matharpada Rd",
    "Amit Naik Marg",
    "Caesar Cross Rd No. 1",
    "Caesar Rd",
    "Swami Vivekanand Rd",
    "Model Co-op Bank",
    "Andheri Subway Rd",
    "Dr. S. Radha Krishnan Rd",
    "MVLU College"
]
route2 = [
    "Home",
    "Matharpada Rd",
    "Amit Naik Marg",
    "Caesar Cross Rd No. 1",
    "Caesar Rd (Alternative)",
    "Swami Vivekanand Rd",
    "Model Co-op Bank",
    "Andheri Subway Rd",
    "Dr. S. Radha Krishnan Rd",
    "MVLU College"
]

# Distances according to the attached route information
route1_distance = 2.2
route2_distance = 2.3

# DISPLAY ROUTES
print("ROUTE 1")
print(" -> ".join(route1))
print("Distance:", route1_distance, "km")
```

```
print("ROUTE 2")
print(" -> ".join(route2))
print("Distance:", route2_distance, "km")

# COMPARE ROUTES
print("ROUTE COMPARISON")
print("Route 1 Distance:", route1_distance, "km")
print("Route 2 Distance:", route2_distance, "km")
if route1_distance < route2_distance:
    print("Route 1 is shorter by",
          round(route2_distance - route1_distance, 2), "km.")
elif route2_distance < route1_distance:
    print("Route 2 is shorter by",
          round(route1_distance - route2_distance, 2), "km.")
else:
    print("Both routes have equal distance.")

# BFS
start = "Home"
goal = "MVLU_College"
result = ""
def BFS(city, cityq, visitedq):
    global result
    if city == start:
        result = result + " " + city
    for eachcity in dict_gn[city].keys():
        if eachcity == goal:
            result = result + " " + eachcity
            return
        if eachcity not in cityq.queue and \
           eachcity not in visitedq.queue:
            cityq.put(eachcity)
            result = result + " " + eachcity
    visitedq.put(city)
    if not cityq.empty():
        BFS(cityq.get(), cityq, visitedq)

# MAIN
def main():
    cityq = Q.Queue()
    visitedq = Q.Queue()
    BFS(start, cityq, visitedq)
```

SHETH L.U.J. AND SIR M.V. DEGREE COLLEGE OF SCIENCE

SUBJECT NAME: <u>T101 ARTIFICIAL INTELLIGENCE Practical</u>

```
    print("BFS RESULT")
    print("BFS Traversal from", start, "to", goal, "is:")
    print(result)
    print("\nDistance Comparison:")
    if route1_distance < route2_distance:
        print("Route 1 =", route1_distance, "km")
        print("Route 2 =", route2_distance, "km")
        print("Therefore, Route 1 is shorter.")
    elif route2_distance < route1_distance:
        print("Route 1 =", route1_distance, "km")
        print("Route 2 =", route2_distance, "km")
        print("Therefore, Route 2 is shorter.")
    else:
        print("Both routes have equal distance.")
main()
```

Output :-